

1. Abstract
2. Introduction
3. Literature Review
4. Materials and datasets
 - a. Dataset used - Cassini

The main experiments for this thesis were run on the dataset used in the paper [Cassini et. al.], where they focused on the semantic segmentation and object detection over the Mesopotmian Floodplain in Iraq. The dataset included 5025 images and masks. The dataset was collected using Bing maps and CORONA data imported into QGIS. The paper evaluated the results using IoU, Precision, Recall, MCC, which are defined as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

$$Recall = \frac{TP}{TP + FN}; Precision = \frac{TP}{TP + FP},$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

This paper compared U-Net versus MA-net, Resnet18 versus Efficientnet-B3 and Dice Loss versus Focal Loss before finally settling on the best model - MA-net, Efficientnet-B3 and Focal Loss, trained for 20 epochs

- b. Models used:
 - i. Segformer: <https://github.com/NVlabs/SegFormer>
Implementation: NVlabs
 - ii. MA-net:
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9201310>
Implementation: segmentation_model in pytorch(?)
 - iii. DeepLab V3+: <https://arxiv.org/pdf/1606.00915>
Implementation: segmentation_model in pytorch(?)

5. Methodology
 - a. Workflow of the PADS profiler
 - b. Metrics captured
6. Results

7. Ablation study
8. Conclusions
9. Future work
10. References

8th July 2026

1. Find new data set for reproducing check
2. Set the formatting for the paper

#####

3. Materials and Datasets

3.1 Overview

This chapter describes the data and the model architectures used throughout this thesis. The experimental subject of the work is a semantic segmentation workload over satellite imagery of the southern Mesopotamian floodplain, and the object of study is the data pipeline that feeds that workload on a shared HPC filesystem. The dataset is therefore treated in two distinct ways in what follows: as a machine learning dataset, characterised by its imagery, masks, class balance and evaluation metrics (Section 3.2); and as an I/O workload, characterised by file count, file size distribution and access pattern (Section 3.3). The second characterisation is the one that motivates PADS-Loader, and it is the one that is usually left implicit in the segmentation literature.

3.2 Dataset: Casini et al. (2023)

The main experiments in this thesis were run on the dataset introduced by Casini et al. (2023) [A human–AI collaboration workflow for archaeological sites detection, Scientific Reports 13:8699], which addresses semantic segmentation and detection of mounded archaeological sites - tells - across the southern Mesopotamian floodplain in Iraq.

3.2.1 Provenance and construction

The dataset originates from a corpus of geo-referenced vector shapes corresponding to the contours of known mound sites within the survey area of the Floodplains Project, which spans approximately 66,000 km². The vector corpus was assembled at the University of Bologna by

consolidating all published archaeological surveys in the region and geo-referencing the sites catalogued therein; it contains 4,934 shapes, each corresponding to a site confirmed by ground-truthing and by study of the associated surface scatter of artefacts.

Because the corpus was compiled as an archaeological reference rather than as a training set, the authors applied two filtering passes before image generation:

1. Removal of the 200 largest sites by area. These sites exceed the area of the square input region and would therefore yield an almost entirely positive segmentation mask, which provides no usable learning signal. Visual inspection confirmed that they follow the shape of extended areas rather than of simple mounds.
2. Removal of 684 sites that were either too small to be a tell or marked as destroyed. The size threshold was set at roughly 1,000 m², corresponding to a circle of 30 m diameter. Sites below this threshold typically correspond to generic annotations for known sites of unknown size or imprecise location.

Image generation was performed in QGIS. For each retained site, a Python script exported a square of side L centred on the site centroid, containing only Bing Maps satellite imagery (accessed through the QuickMapService plugin). The same square was exported a second time without the base map but with the site contour rendered as a solid filled shape, giving the corresponding binary ground-truth mask. The task is therefore binary: one class for mounded (tell) sites, one for everything else.

Two spatial configurations were produced: $L = 1000$ m ("1k") and $L = 2000$ m ("2k"). Imagery was exported at approximately 1 pixel per metre (1024 px for 1000 m, and double that for the larger extent) and subsequently downsampled by a factor of two to reduce computational cost, which the original authors report as having low impact on accuracy. At training time a square of side $L/2$ is randomly cropped from the exported tile; this both prevents the model from learning a positional prior in which sites always appear at the image centre and acts as data augmentation. Further augmentation consists of random rotation, mirroring, and small shifts in brightness and contrast, resampled at each training iteration.

Finally, 1,155 images with empty masks - containing no site to predict - were added, sampled from locations proposed by the archaeologists: highly urbanised areas, intensive agriculture, flood-prone terrain such as artificial lakes and basins, and rocky hills and mountains. These negatives are what force the model to learn what a tell is not. The final dataset consists of 5,025 image/mask pairs, split 90% training / 10% held-out test, stratified over the empty images, with a further 10% of the training partition randomly reserved as a validation set.

A CORONA-augmented variant of the dataset also exists, in which declassified historical CORONA satellite imagery is imported into QGIS and cropped identically to the Bing tiles, giving a second input channel group. Because CORONA was used only as a complement in the original work, sites destroyed after the 1970s were excluded from that analysis.

Dataset variant used in this thesis. All experiments reported here use the Bing 1k configuration unless otherwise stated. This choice is deliberate: the 1k configuration is the smallest of the published variants and therefore the least favourable to a data-staging argument - a pipeline that is I/O-bound on 1k tiles will be at least as I/O-bound on 2k tiles or on the CORONA-augmented input. Section 6 (ablation) revisits the larger input configurations.

3.2.2 Class imbalance

The segmentation target is severely imbalanced: mound pixels constitute a small minority of any given tile, and 1,155 of the 5,025 tiles contain no positive pixels at all. This is the reason the original work adopts Focal Loss over Dice Loss, and it is the reason the primary reported metric is IoU rather than pixel accuracy. It is also, importantly for this thesis, the reason the workload is decode-heavy relative to its useful signal: a substantial fraction of the bytes read from the filesystem contribute nothing but negative pixels.

3.2.3 Evaluation metrics

The original paper evaluates using Intersection-over-Union at the pixel level, and Accuracy, Precision, Recall and the Matthews Correlation Coefficient at the site-detection level. These are defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

$$Recall = \frac{TP}{TP + FN}; Precision = \frac{TP}{TP + FP},$$

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}.$$

The distinction between the two levels matters. IoU measures how well a predicted contour overlaps an annotated one; it does not measure how many sites were found. The original authors note that IoU can improve while detection accuracy degrades, as occurs between their Model 5 and Model 6: a more conservative model produces fewer positive predictions, shrinking the union term and raising IoU, while missing sites and lowering recall. Any model comparison in this thesis that relies on IoU alone should be read with that caveat in mind.

3.2.4 Reference results

Casini et al. report the following IoU scores on the held-out test set. Standard deviations arise from repeated testing to average out the random crop.

Model	Input	Filter	IoU (%)	Std. dev
Model 1	Bing 1k	-	74.17	0.38
Model 2	Bing 1k	✓	78.10	0.54
Model 3	Bing + CORONA 1k	-	74.06	0.39
Model 4	Bing 2k	-	79.77	0.34
Model 5	Bing 2k	✓	81.54	0.35
Model 6	Bing + CORONA 2k	✓	83.45	0.18

At the detection level, Model 5 achieves 62.57% accuracy under automatic evaluation and 80.40% after human-in-the-loop adjustment for mislabelled (no-longer-visible) sites; Model 6 achieves 60.08% and 79.13% respectively. Their configuration search compared U-Net against MA-Net, ResNet-18 against EfficientNet-B3, and Dice Loss against Focal Loss, settling on MA-Net + EfficientNet-B3 + Focal Loss, trained for 20 epochs, as the best configuration; they observe that the differences between configurations were small, within a few percentage points, and plausibly attributable to fluctuation from the random augmentation.

These figures serve two purposes here. First, they establish an accuracy floor: any optimisation to the data pipeline proposed in this thesis must be shown not to degrade segmentation quality relative to this reference. Second, the observation that architectural choice moves IoU by only a few points motivates the framing of this work - if the model is not where the remaining headroom is, the pipeline may be.

3.3 The dataset as an I/O workload

The above is the standard description of the dataset. For the purposes of this thesis a second description is required, because the property of the dataset that PADS-Loader responds to is not its class balance but its file layout.

The Bing 1k dataset consists of 5,025 image files and 5,025 corresponding mask files stored as individual objects on the Iris GPFS shared filesystem. Each training step therefore issues, per sample, an independent `open/stat/read/close` sequence against a parallel filesystem that is optimised for large sequential transfers rather than for many-small-file metadata operations. The relevant properties are:

- Many small files. Metadata cost per sample is close to constant regardless of how few bytes are actually read, so per-file overhead does not amortise.
- Paired access. Every image read is accompanied by a mask read at a different path, doubling the metadata traffic per sample.

- Shared contention. Iris GPFS is a shared resource; observed read latency is not a property of the dataset alone but of concurrent jobs on the cluster, and is therefore non-stationary across a run.
- Deterministic access order. Once the epoch permutation is fixed, the sequence of samples - and hence the sequence of files - is known in advance. Future data access is predictable, which is precisely the precondition that makes proactive prefetching and shard staging possible rather than speculative.

The final point is the hinge of the entire method. The dataset is small enough to fit on node-local scratch and its access pattern is known ahead of time; the fact that the baseline pipeline nonetheless leaves the GPU idle is a property of the loader, not of the data.

3.4 Models

Three segmentation architectures are used, spanning a CNN encoder–decoder, an attention-augmented CNN, and a transformer. All three were trained under an identical data pipeline so that pipeline effects can be isolated from architectural effects.

3.4.1 MA-Net

MA-Net (Multi-scale Attention Network; Fan et al., IEEE Access 8, 2020) can be read as an extension of the U-Net encoder–decoder with a self-attention mechanism inserted in the latent space, allowing the network to weight latent features according to content. Originally developed for liver and tumour segmentation in medical imaging, it has since been applied to remote sensing. It is included here because it is the architecture selected as best-performing by Casini et al., and therefore constitutes the reproduction baseline for this thesis. Implementation: `segmentation_models_pytorch` (Iakubovskii, 2019), with an EfficientNet-B3 encoder pretrained on ImageNet.

3.4.2 DeepLabV3+

DeepLabV3+ (Chen et al., arXiv:1606.00915 and successors) uses atrous (dilated) convolution and atrous spatial pyramid pooling to capture multi-scale context without loss of spatial resolution, combined with a decoder module that recovers object boundaries. It is included as a widely-used, high-capacity CNN baseline and represents the heaviest of the three in both parameter count and per-step GPU time. Implementation: `segmentation_models_pytorch`.

3.4.3 SegFormer

SegFormer (Xie et al., NeurIPS 2021; NVlabs reference implementation) pairs a hierarchical, positional-encoding-free transformer encoder with a lightweight all-MLP decoder. It is included as the transformer baseline and, critically for this thesis, as the least GPU-bound of the three architectures: its low per-step compute makes it the configuration in which data-loading stalls are most exposed. Implementation: NVlabs SegFormer.

3.4.4 Measured characteristics (Bing 1k, default loader)

The following are measured on the Iris cluster on a single V100, with the baseline PyTorch DataLoader (default settings):

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
IoU (%)	67	68	35.9
Epochs	34	21	46
Batch size	32	32	32
CPU time (s)	3.922	4.512	4.639
GPU time (ms)	416.305	644.360	830.446

With loader parameters tuned (num_workers=4, pin_memory=True):

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
IoU (%)	67	73	43
Epochs	34	22	43
Batch size	32	32	32
CPU time (s)	3.922	4.737	5.245
GPU time (ms)	416.305	623.832	910.236

Two observations from these tables set up Section 4. First, the ratio of CPU time to GPU time per step is large enough across all three architectures that the data path is a plausible bottleneck rather than a rounding error. Second, merely changing two loader flags moves MA-Net IoU by five points and DeepLabV3+ by seven - a swing comparable in magnitude to the entire architectural search in the original paper. That the loader configuration is competitive with the architecture choice as a source of variation is the empirical hook for this work.

3.5 Hardware and software environment

All experiments were run on the Iris cluster at the University of Luxembourg, on GPU nodes equipped with NVIDIA V100 accelerators, scheduled via Slurm. Storage is provided by a shared GPFS filesystem, with node-local scratch available for staging. The software stack is containerised with Apptainer to guarantee a reproducible environment across nodes; the container image bundles PyTorch, PyTorch Lightning, `segmentation_models_pytorch`, and `albumentations`.

4. Methodology

4.1 Motivation: when the DataLoader becomes the bottleneck

Deep learning training on HPC is conventionally treated as a GPU problem. The profiling results reported in Section 3.4, together with a targeted profiling pass over the baseline pipeline, indicate that for this workload the GPU is frequently not the constraint. Four findings motivate PADS-Loader:

1. GPU starvation. With `num_workers=0`, the V100 sits idle between batches. The accelerator is waiting on the data path, not on the model.
2. Shared filesystem pressure. Thousands of small image and mask files generate high metadata overhead on Iris GPFS. The cost is dominated not by bytes transferred but by per-file operations against a shared parallel filesystem.
3. Static configuration. `num_workers`, `prefetch_factor` and `pin_memory` are set once at loader construction and never adapt. They cannot respond to a bottleneck that shifts - as it does, when contention on a shared filesystem varies over the lifetime of a job.
4. Deterministic access. Because the sample order is fixed once the epoch permutation is drawn, the sequence of future file accesses is known. Prefetching is therefore not speculative: it can be exact.

Taken together, these give the research question of this thesis:

Can a profiler-adaptive DataLoader that combines bottleneck classification, tar-shard staging, and predictive prefetching improve GPU utilisation and tile throughput for large-scale satellite-image segmentation on HPC systems?

which decomposes into four sub-questions:

- RQ1. When does the DataLoader become the bottleneck on Iris GPFS?
- RQ2. Do tar shards reduce many-small-file overhead?
- RQ3. Does predictive scratch staging outperform fixed WebDataset streaming?
- RQ4. Can prefetch depth be auto-tuned from measured timings?

4.2 PADS-Loader: system overview

PADS-Loader (Profiler-Adaptive Data Staging) is a drop-in replacement for the PyTorch DataLoader that observes its own behaviour during a short warm-up window, classifies the dominant bottleneck from the observed timings, selects a staging policy accordingly, and then executes that policy with predictive prefetching for the remainder of the run.

The pipeline has four stages:

1. WARM-UP PROFILING → 2. BOTTLENECK CLASSIFICATION → 3. POLICY SELECTION → 4. PREDICTIVE STAGING
50–100 batches I/O · decode · H2D · compute loose / shard / stage shard
prefetch to scratch

The design intent is that stages 1–3 cost a bounded, one-off overhead (tens of seconds on a job measured in hours) and that stage 4 recovers that cost many times over. The loader is deliberately conservative by default: if no bottleneck is detected, it degrades to the loose policy, which is the ordinary PyTorch DataLoader.

4.2.1 Stage 1 - Warm-up profiling

For the first 50–100 batches, PADS-Loader instruments the data path and records per-batch timings without altering behaviour. The signals measured are:

Signal	Meaning
T_data	Time spent waiting for the next batch to become available
T_decode	Image and mask decode time, plus augmentation transforms
T_h2d	Host-to-device (CPU→GPU) transfer time
T_gpu	Model forward + backward time
GPU utilisation	Accelerator occupancy over the window
Queue fullness	DataLoader output-queue depth - a full queue means the GPU is the constraint; a persistently empty one means the loader is
Shard size	Whether the next tar shard can be staged efficiently given its size
Scratch availability	Whether node-local scratch has sufficient free capacity for staging

The final two are feasibility signals rather than performance signals: they determine whether a policy is available, not whether it is desirable.

[TODO: specify how each timer is implemented - `torch.cuda.Event` for `T_gpu` and `T_h2d`, wall-clock around the iterator for `T_data`, explicit instrumentation inside `__getitem__` for `T_decode`. State the aggregation used (median vs mean; robustness to first-batch warm-up outliers) and the warm-up batches discarded before statistics are taken.]

4.2.2 Stage 2 - Bottleneck classification

The profiled signals are mapped onto a dominant bottleneck class. The classification is not a black box: it is a small set of rules over the measured timings, chosen so that the decision is inspectable and reproducible.

Detected condition	Interpretation	Action
<code>T_data</code> low, <code>T_gpu</code> high, GPU utilisation high	Compute-bound; the pipeline is already keeping up	Loose
Many open/stat/read calls; high metadata overhead	Many-small-file overhead dominates	Shard
Shared filesystem read latency high; scratch available	Shared-FS latency dominates and can be avoided	Stage
Many-small-file overhead and slow shared FS	Both pathologies present	Shard + Stage
<code>T_decode</code> dominates	CPU-side decode is the constraint	Increase workers / cache decoded samples

Policies are not mutually exclusive. Scratch staging and an increased worker count may both be active simultaneously when filesystem I/O and decode are co-bottlenecks. The classifier therefore emits a set of actions rather than a single label.

4.2.3 Stage 3 - Policy selection

Three staging policies are defined:

Policy	What it does	Best when
--------	--------------	-----------

Loose	Reads individual image and mask files directly from GPFS	Simple baseline; small or node-local dataset; workload already compute-bound
Shard	Reads samples sequentially from tar archives, amortising per-file metadata cost over many samples	Many-small-file overhead is the problem
Stage	Copies future shards to node-local scratch ahead of use, removing the shared filesystem from the critical path	Shared filesystem latency is the problem

Loose is the baseline and the fallback. Shard attacks metadata overhead by converting thousands of open calls into a handful of large sequential reads. Stage attacks latency and contention by moving data off the shared filesystem entirely, exploiting the fact that the dataset fits in node-local scratch. The two compose: shards may be both formed and staged.

4.2.4 Stage 4 - Predictive staging

Because the epoch permutation is fixed at the start of the epoch, the identity of the next k shards is known exactly. PADS-Loader exploits this by maintaining a background staging thread that copies shard $i + d$ to node-local scratch while shard i is being consumed, where d is the prefetch depth.

The prefetch depth d is not a hyperparameter supplied by the user but is derived from the measured timings: it is set such that the time to stage a shard is covered by the time to consume the shards ahead of it, subject to the scratch capacity constraint that no more than d shards may be resident at once. This is the mechanism by which RQ4 - can prefetch depth be auto-tuned from measured timings? - is answered affirmatively by construction, and it is what distinguishes PADS-Loader from WebDataset, which streams shards at a fixed, statically configured depth without reference to whether that depth is sufficient to hide the transfer.

4.3 Metrics captured

Two families of metric are reported.

Pipeline metrics (the object of the optimisation):

- GPU utilisation - mean accelerator occupancy over the run, the headline indicator of whether starvation has been eliminated.
- Tile throughput - samples processed per second; the end-to-end figure of merit.
- Time-to-epoch - wall-clock seconds per epoch, and total time-to-target-loU.

- $T_{\text{data}} / T_{\text{decode}} / T_{\text{h2d}} / T_{\text{gpu}}$ breakdown - retained for the full run, not just warm-up, so that the shift in the bottleneck under each policy is visible rather than merely its disappearance.
- Filesystem operation counts - open/stat/read calls issued against GPFS, which is the direct measure of whether sharding achieved what it was intended to achieve (RQ2).
- Scratch traffic and residency - bytes staged, staging hit rate, peak scratch occupancy.
- Profiling overhead - the cost of the warm-up window itself, reported explicitly so that the net benefit is honest.

Model metrics (the constraint, not the objective):

- IoU on the held-out test set, against the Section 3.2.4 reference.
- Epochs to convergence, and detection-level Accuracy / Precision / Recall / MCC where a comparison to Casini et al. is being made.

The reason for reporting both is that this thesis makes a negative claim about accuracy as well as a positive claim about throughput: PADS-Loader is only useful if it accelerates the pipeline without degrading segmentation quality. A throughput improvement obtained at the cost of reduced shuffling randomness, and therefore of a worse model, is not an improvement. Any observed IoU deviation greater than the ± 0.5 pp standard deviation reported in the reference results must be treated as a regression to be explained, not as noise.

4.4 Evaluation plan

PADS-Loader is compared against four alternatives, all on the identical model, dataset and hardware, with only the data path varying:

#	Method	What it isolates
1	Baseline PyTorch DataLoader - loose files, default settings	The unmodified starting point
2	Tuned PyTorch DataLoader - best num_workers, pin_memory, prefetch_factor	How much is recoverable by manual tuning alone; PADS-Loader must beat this, not merely the naive baseline
3	WebDataset - fixed tar-shard streaming, no adaptation	Isolates the benefit of sharding from the benefit of adaptation (RQ2 vs RQ3)
4	Scratch baseline - loose files copied manually to node-local scratch up front	Isolates the benefit of staging from the benefit of predicting what to stage; the ceiling for any policy that removes GPFS from the path

5 PADS-Loader (proposed) - adaptive - staging + predictive prefetch

The comparison set is constructed so that each of the four sub-questions is answered by a specific contrast rather than by the aggregate result:

- RQ1 (when does the loader become the bottleneck?) is answered by the profiling breakdown across methods 1 and 2 and across the three architectures - SegFormer, with the lowest T_{gpu} , is the configuration in which the loader bottleneck should appear first.
- RQ2 (do tar shards help?) is answered by 3 vs 1, and by the filesystem operation counts.
- RQ3 (does predictive staging beat fixed streaming?) is answered by 5 vs 3, with 4 as the upper bound.
- RQ4 (can prefetch depth be auto-tuned?) is answered by 5 vs 3, and by a sweep of fixed depths in the ablation of Section 6 - the auto-tuned depth should land at or near the best fixed depth, without having been told it.

Method 4 deserves emphasis: it is the honest ceiling. If manually pre-staging the entire dataset to scratch matches PADS-Loader, then adaptivity has bought nothing for this dataset, and the contribution reduces to the observation that the dataset fits in scratch. PADS-Loader's claim is that it reaches that ceiling without requiring the user to know in advance that scratch is the answer, and that it continues to work when the dataset does not fit - which is the regime the design is actually aimed at, and which Section 6 probes by synthetically constraining scratch capacity.